

Projet final — Soutenance

React.js & TypeScript · Bachelor 2 · Application complète en React · Groupe · Formateur : PREVOST Nathan

Objectif. Proposer une solution à travers une application React et TypeScript complète, connectée à une API et déployée en ligne pour répondre à un problème

Projet à déposer sur moodle au plus tard la veille de la soutenance.

À déposer : le code complet du projet (dépôt Git accessible, ou archive du code source sans node_modules) · le support de présentation utilisé lors de la soutenance · l'adresse de l'application déployée · le README indiquant la répartition du travail entre les membres. Un dépôt privé doit être partagé avec le formateur.

Attendue pour l'oral

Une présentation complète avec démo

Attendue avant l'oral

Projet complet (code) avec présentation

Temps de passage

Durée	Contenu
10 min	Présentation et démonstration en ligne : sujet, source des données, puis parcours complet de l'application — liste, recherche, détail, formulaire, état global, cas d'erreur.
5 à 10 min	Questions.

Organisation : Groupes de 4-5 personnes · La démonstration se fait depuis l'adresse déployée, jamais depuis le serveur de développement · Prévoir un partage de connexion en secours.

Compétences à démontrer

Domaine	Ce qui doit être visible ou expliqué
TypeScript	Types primitifs, interfaces, unions pour un état fini, au moins un générique écrit par le groupe. Aucun any non commenté. tsc --noEmit passe.
Configuration	Projet Vite + react-ts. Savoir expliquer trois options du tsconfig.json. HMR en développement, build de production sans erreur.
Composants	Huit composants minimum. Deux composants réutilisables, dont un avec children. Props typées partout.
Hooks	useState, useEffect, useContext, useReducer employés à bon escient. Deux hooks personnalisés factorisant une logique réelle.
Formulaires	Champs contrôlés, validation côté client, messages d'erreur au niveau des champs, soumission bloquée tant que le formulaire est invalide.
Routage	React Router v6 car vu en cours : quatre routes minimums, une route à paramètre, une navigation programmatique, une mise en page avec Outlet, une page 404.
État global	Un Context partagé entre composants éloignés, dont l'état est géré par useReducer. Immuabilité respectée partout.
API et asynchrone	Données chargées par fetch. Les trois états traités et visibles : chargement, erreur, succès. Nettoyage des effets, situations de course traitées.
Performance	Savoir mesurer avant d'optimiser avec le Profiler. memo, useMemo, useCallback employés seulement là où le gain est constaté.
Tests	Quatre tests minimums avec Vitest et Testing Library, dont un portant sur un comportement conditionnel. Assertions réelles.
(BONUS) Déploiement	Application accessible en HTTPS à une adresse publique. README permettant à un tiers d'installer et lancer le projet.

À vérifier avant de passer

- L'application se charge depuis son adresse publique, sur un navigateur neutre.
- Vous savez provoquer une erreur réseau en direct et montrer ce qui s'affiche.
- Le README indique la répartition du travail, domaine par domaine.